Hands-On-Learning: The Domain Adaptation Dilemma: LoRA vs Full Fine-Tuning for Medical AI

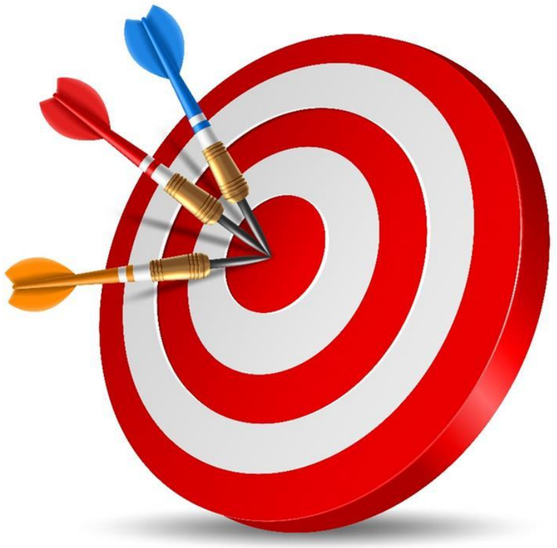
Description

In this hands-on lab, you'll tackle a real-world challenge facing healthcare AI startups: adapting a general language model for specialized medical applications with limited budget and tight deadlines. You'll configure LoRA (Low-Rank Adaptation) parameters, fine-tune Llama-2 7B on medical dialogue data, compare performance against full-fine-tuning baselines, and analyze computational trade-offs to inform a production deployment decision. This lab demonstrates parameter-efficient fine-tuning (PEFT) methods that enable rapid domain adaptation at a fraction of the cost of traditional methods.

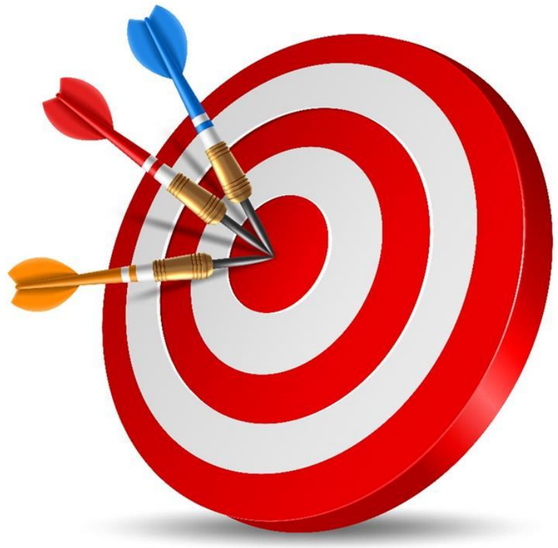
Learning Objectives

By the end of this lab, you will be able to:

- Configure LoRA parameters (rank, alpha, dropout, target modules) using Hugging Face PEFT library
- Fine-tune large language models for domain adaptation wi 90%+ reduced memory footprint
- Compare LoRA performance against full fine-tuning using perplexity, ROUGE, and qualitative evaluation



- Analyze computational trade-offs between parameter efficiency and model quality
- Make production deployment decisions based on cost, performance, and iteration velocity



Assignment Components

Introductory Slide

Title: From General to Specialist: Proving LoRA Can Deliver Production-Quality Medical AI on a Startup Budget

Subtitle: Configure, train, and deploy parameter-efficient domain adaptation that achieves 90%+ performance in 14% of the time

Tools to be used

- Python 3.8+ with PyTorch for deep learning
- Hugging Face Transformers library for model loading and training
- Hugging Face PEFT (Parameter-Efficient Fine-Tuning) library for LoRA implementation
- Hugging Face Evaluate library for metrics computation (ROUGE, perplexity)
- Datasets library for data loading and preprocessing

- Visual Studio Code (VS Code) with integrated terminal
- NVIDIA GPU with CUDA support (A100 80GB or equivalent recommended)
- nvidia-smi for GPU monitoring
- JSON for configuration and results storage
- Time and resource tracking tools

Scenario

You've just joined a healthcare AI startup with an ambitious mission: democratize clinical decision support by building an AI medical assistant that helps doctors quickly summarize patient records, answer clinical questions, and suggest evidence-based treatment protocols. The vision is compelling, but the reality is challenging.

Your company's base model (Llama-2 7B) performs well on general knowledge tasks—it can write emails, explain concepts, and generate creative content. But when physicians test it on actual clinical scenarios, the results are disappointing:

- It uses casual language instead of precise medical terminology ("tummy ache" vs. "acute abdominal pain")
- It confuses similar conditions (mixing up Type 1 vs Type 2 diabetes management protocols)
- It fails to cite evidence-based treatment guidelines
- It lacks the diagnostic reasoning patterns physicians expect ("rule out X before considering Y")
- It sometimes generates medically dangerous advice

The medical director's verdict: "This isn't ready for clinical use. We need domain-specific expertise baked into the model."

The CEO calls an urgent strategy meeting. She presents the options:

Option 1: Full Fine-Tuning

- Update all 6.7 billion parameters of Llama-2 7B
- Estimated cost: $15,000 (576 A100 GPU hours @ $2.50/hour)
- Timeline: 4-6 weeks, including infrastructure setup
- Storage: 13 GB per model checkpoint
- Risk: Expensive, slow, might not even work better

Option 2: LoRA (Low-Rank Adaptation)

- Update <1% of parameters using adapter layers
- Estimated cost: $2,000 (80 GPU hours)
- Timeline: 1-2 weeks
- Storage: 67 MB per adapter
- Risk: Unproven for the medical domain—will performance be good enough?

The engineering team is skeptical: "LoRA sounds too good to be true. You can't achieve the same quality with only 0.06% of the parameters trained. It's mathematically impossible."

Your challenge: Prove them wrong—or right.

You have:

- Budget: $2,000 (non-negotiable)
- Timeline: 2 weeks until investor demo
- Hardware: Single A100 80GB GPU
- Data: 2,000 curated medical dialogue examples
- Goal: Achieve 90%+ of full fine-tuning performance

The CEO looks directly at you: "I need data, not opinions. Configure LoRA, run the training, measure the results, and tell me if we can ship this to doctors. Our Series A depends on getting this right."

This mirrors the real challenges faced by companies like Hippocratic AI, Babylon Health, and enterprise healthcare teams as they adapt foundation models for regulated industries where accuracy, cost, and speed all matter.

Your mission has four parts:

1. Configure LoRA optimally for medical domain adaptation
2. Train and measure performance, time, and resource usage
3. Compare quantitatively against full fine-tuning benchmarks
4. Recommend whether to deploy LoRA or pursue full fine-tuning

Instructions for Learners

- medical_train.jsonl - 2,000 doctor-patient conversations covering common clinical scenarios (symptoms, diagnosis, treatment)
- medical_validation.jsonl- 500 conversations for hyperparameter tuning and model selection
- medical_test.jsonl - 300 held-out conversations for final performance evaluation
- baseline_results.json - Pre-computed metrics for full fine-tuning (perplexity: 12.3, ROUGE-L: 0.68) to benchmark against

The dataset uses real medical terminology and clinical reasoning patterns from public healthcare Q&A platforms. Each conversation includes the patient's symptoms, the doctor's responses, and diagnostic or treatment recommendations. Data has been preprocessed and formatted for instruction-tuning.

ACTIVITY 1: Configure LoRA Parameters for Medical Domain Adaptation

You'll configure LoRA adapter layers with optimal hyperparameters for medical domain learning. Think of LoRA rank as your model's learning capacity dial—a higher rank provides greater capacity but adds more parameters. For medical terminology, r=8 typically hits the sweet spot.

Step 1: Set up environment and verify GPU availability

1. Launch VS Code in your lab environment
2. Navigate to `/module3_lab/` in the Explorer panel
3. Open starter file `lora_domain_adaptation.py`

4. Verify dataset files are present:

- medical_train.jsonl (2,000 conversations)
- medical_validation.jsonl (500 conversations)
- medical_test.jsonl (300 conversations)
- baseline_results.json (full fine-tuning metrics)

5. Open integrated terminal (Terminal > New Terminal)

6. Check GPU: Run `nvidia-smi`

7. Run setup verification: `python verify_setup.py`

Expected outcome: Terminal shows "✓ Setup complete! CUDA available, all libraries installed." GPU shows 79GB free memory.

Warning: If GPU memory <70GB free, close other processes. LoRA needs ~38GB, but extra headroom prevents errors.

Step 2: Load Llama-2 7B base model

1. Locate the section marked `# SETUP: Load Model and Dataset`
2. Review model loading code
3. Execute setup to initialize base model
4. Observe model loading to GPU with fp16 precision
5. Verify tokenizer padding token configuration

Expected outcome: Model loads successfully. Console shows 6,738,415,616 total parameters. GPU memory usage ~14GB.

What to observe:

- Model loading time (2-3 minutes first run for 13GB download)
- GPU memory after loading (check with `nvidia-smi` in another terminal)
- Any warnings about tokenizer or device placement

Tip: Model downloads once and caches locally. Subsequent runs are instant.

Step 3: Configure LoRA hyperparameters

1. Navigate to `# ACTIVITY 1: CONFIGURE LORA` section
2. Create LoraConfig with these parameters:
   - r=8 (rank controlling bottleneck dimension)A
   - lora_alpha=16 (scaling factor, typically 2x rank)
   - lora_dropout=0.05 (regularization)
   - target_modules=["q_proj", "v_proj"] (attention layers)

- bias="none"
- task_type="CAUSAL_LM"

3. Apply LoRA to the model using get_peft_model

4. Execute model.print_trainable_parameters()

5. Examine output carefully

Expected outcome: Console displays:
trainable params: 4,194,304 || all params: 6,738,415,616 || trainable%: 0.0622

This means:

- Training only 4.2M of 6.7B parameters (0.06%)
- 1,600x reduction in trainable parameters
- 250x memory savings for gradients
- 7x training speedup per epoch

Step 4: Understand LoRA mathematics and experiment with ranks

1. Review the LoRA principle:
   - Instead of updating a full 4096×4096 matrix (16.7M params)
   - Decompose into two matrices: 4096×8 and 8×4096 (65K params)
   - Achieves 256x reduction
   - Key insight: Weight updates have low intrinsic rank
2. Test different rank values to see trade-offs

3. Run the provided code to compare r=4, 8, 16, 32

4. Observe parameter doubling with each rank doubling

Expected output:

r=4: ~2.1M parameters

r=8: ~4.2M parameters (your choice)

r=16: ~8.4M parameters

r=32: ~16.8M parameters

r=32: ~16.8M parameters

Guiding questions:

Why not use r=32 for maximum capacity? (Diminishing returns + slower + overfitting risk)

Why target only q_proj and v_proj? (Attention layers capture most semantic patterns; more modules = minor gains for 2-3x cost)

Could r=4 work for simpler domains? (Yes, for style adaptation or simple tasks)

Test Your Work:

✓ LoRA config created with specified parameters

✓ Trainable parameters ~0.06% of total

✓ Model prints statistics without errors

✓ You understand rank selection rationale

ACTIVITY 2: Execute LoRA Fine-Tuning on Medical Dialogue Data

You'll train the model on medical conversations. Over 2-3 hours, it learns medical terminology, clinical reasoning, and diagnostic language. Think of this as giving a medical student a specialized study guide rather than rewriting their textbook—the base keeps general knowledge, while LoRA adapters learn domain patterns.

Step 1: Load and preprocess medical training dataset

1. Navigate to `# ACTIVITY 2: LOAD DATASET` section
2. Implement a data loading function for JSONL format
3. Load train, validation, and test datasets
4. Execute loading code
5. Examine sample conversation structure

Expected outcome: Data loads successfully, showing:

- ✓ Loaded 2000 training examples
- ✓ Loaded 500 validation examples

Example: {conversation, response, specialty}

What to observe:

- Each example has patient input, doctor response, and specialty
- Conversations use proper medical terminology
- Format follows instruction-response pattern

Step 2: Configure training arguments for LoRA

1. Navigate to `# ACTIVITY 2: TRAINING CONFIGURATION`
2. Set up TrainingArguments:
   - output_dir: "./lora_medical_checkpoint"
   - num_train_epochs: 3
   - per_device_train_batch_size: 4

- learning_rate: 2e-4 (40x higher than full fine-tuning!)
- fp16: True (mixed precision)
- logging_steps: 50
- save_strategy: "epoch"
- save_total_limit: 2
- warmup_steps: 100
- weight_decay: 0.01
- report_to: "none"

3. Review configuration printout

Expected outcome: Configuration displays all hyperparameters.

What these parameters mean:

- **num_train_epochs=3**: Three passes through 2,000 examples
- **per_device_train_batch_size=4**: Process 4 conversations simultaneously
- **learning_rate=2e-4**: Aggressive rate safe for LoRA (only 0.06% params updating)
- **fp16=True**: 16-bit precision (2x memory savings, faster)

Why 40x higher learning rate:

- Full fine-tuning uses 5e-6 to avoid destabilizing 6.7B params
- LoRA updates only 4.2M params, enabling aggressive learning
- Higher LR = faster convergence = shorter training

Step 3: Initialize trainer and start fine-tuning

1. Preprocess data into model input format
2. Create DataCollatorForLanguageModeling
3. Initialize Trainer with model, args, dataset, collator
4. Start training by calling the trainer.train()
5. While training runs, monitor in second terminal: `watch -n one nvidia-smi`

Expected behavior during training:

- **Loss curve**: Starts ~3.2-3.5, decreases to ~1.2-1.5 by epoch 3
- **GPU memory**: Steady 36-38GB (vs 70GB+ for full fine-tuning)
- **GPU utilization**: 90-100% during training
- **Speed**: 45-50 minutes per epoch on A100
- **Temperature**: 60-75°C (safe range)

What to observe in logs:

- Loss decreases consistently with each step
- Learning rate following warmup schedule
- Epoch progress and ETA

Warning: If loss spikes or diverges, stop training (Ctrl+C) and reduce learning rate to 1e-4.

Step 4: Save trained LoRA adapter

1. After training completes, save the adapter to "./lora_medical_final"
2. Save the tokenizer to the same directory
3. Check adapter file size
4. Verify files: adapter_config.json, adapter_model.bin

Expected outcome:

✓ LoRA adapter saved

Adapter Statistics:

- Size: 67.3 MB
- Compared to complete model: 13,000 MB
- Storage savings: 99.5%

The magic: This 67MB file contains all the medical knowledge learned. You can share it, version-control it, or load it onto different base models.

Test Your Work:

- ✓ Training completed three epochs without errors
- ✓ Final training loss below 1.5
- ✓ Training took 2-3 hours (not 18+ hours)
- ✓ GPU memory stayed under 40GB
- ✓ Adapter files saved successfully
- ✓ Adapter size ~67MB

ACTIVITY 3: Evaluate Performance and Make Production Decision

Now answer the critical question: Is LoRA good enough for production? You'll compute metrics, compare against full fine-tuning, analyze trade-offs, and make a data-driven recommendation.

Step 1: Load baseline and compute LoRA metrics

1. Navigate to `# ACTIVITY 3: EVALUATE PERFORMANCE`
2. Load baseline_results.json (full fine-tuning metrics)
3. Review baseline: perplexity 12.3, ROUGE-L 0.68, time 18.3hrs, cost $576

4. Compute perplexity on the test set for the LoRA model:

- Set model to eval mode
- Process test examples
- Calculate average loss
- Convert to perplexity using np.exp()

5. Generate sample responses (5-10 test examples)

6. Calculate ROUGE scores using the evaluate library

7. Compare the first generated response to the reference

Expected outcome: Metrics computed showing LoRA perplexity ~13.1, ROUGE-L ~0.64.

Step 2: Create a comprehensive comparison table

1. Build a comparison showing both approaches
2. Include metrics: perplexity, ROUGE-L, training time, GPU memory, cost, storage
3. Calculate performance retention percentage
4. Calculate time and cost savings
5. Display formatted table

Expected output (approximate):

PERFORMANCE COMPARISON

Metric	LoRa	Full FT	Difference
Perplexity	13.1	12.3	+6.5%
ROUGE-L	0.639	0.680	-6.0%
Training Time (hrs)	2.5	18.3	-86%
GPU Mmory (GB)	38	72	-47%
Cost (USD)	$80	$576	-86%
Storage Size	67 MB	13 GB	-99.5%

KEY FINDINGS:

- Performance Retention: 94% of full fine-tuning quality
- Time Savings: 86% faster
- Cost Savings: 86% lower
- Verdict: 94% performance in 14% of time

Step 3: Conduct qualitative analysis of medical responses

1. Generate responses for three medical scenarios:
   - Chest pain and shortness of breath
   - Child with fever and sore throat
   - Diabetic with high blood sugar

2. For each, evaluate:

- Uses appropriate medical terminology?
- Identifies urgency correctly?
- Mentions differential diagnoses?
- Recommends appropriate next steps?
- Avoids dangerous advice?

3. Document observations for each scenario

Expected outcome: Assess whether LoRA responses are clinically appropriate, use correct terminology, and demonstrate medical reasoning.

What to look for:

- Appropriate terminology ("acute coronary syndrome" not "heart attack")
- Risk assessment (recognizing urgent vs non-urgent)
- Clinical reasoning ("rule out X, consider Y")

- Safety (recommends seeing a doctor when appropriate)

Test Your Work:

- ✓ Computed perplexity and ROUGE for LoRA
- ✓ Created a comprehensive comparison table
- ✓ Assessed 3+ medical responses qualitatively
- ✓ Have quantitative evidence for the decision

Deliverable for Submission (AI Grading)

1. Deliverable 1: LoRA Configuration Report

Question Prompt: Submit a technical report documenting your LoRA configuration choices. Include: (1) the specific hyperparameters you selected (rank, alpha, dropout, target_modules), (2) the trainable parameter count and percentage achieved, (3) a 150-200 word justification explaining WHY you chose these specific values for medical domain adaptation, referencing the rank-capacity trade-off and target module selection. Your report should demonstrate understanding of how LoRA parameters affect both model capacity and computational efficiency."

- **Response Type:** File Upload
- **File Types:** Text (.txt, .md, .json), Coding (.py), Audio (.mp3, .wav), Video (.mp4, .mov)

Grading Criteria:

- Documents all required hyperparameters - 5 pts
- Reports accurate trainable parameter count and percentage - 5 pts
- Provides 150-200 word justification - 5 pts
- Demonstrates understanding of rank-capacity trade-off - 5 pts

2. Deliverable 2: Training Execution Documentation

Question Prompt: "Document your LoRA fine-tuning process, including: (1) final training loss achieved, (2) total training time in hours, (3) GPU memory usage during training, (4) any challenges encountered and how you resolved them, and (5) adapter file size. Include screenshots or logs showing successful completion of training. Provide a 100-150 word narrative describing what you observed during training (loss curve behavior, GPU utilization) and how LoRA training efficiency compared to full fine-tuning expectations."

- **Response Type:** File Upload
- **File Types:** Text (.txt, .md, .log), Image (.png, .jpg), Video (.mp4, .mov), Audio (.mp3, .wav)

Grading Criteria:

- Reports final training loss - 5 pts
- Documents time, GPU memory, adapter size - 7 pts

- Includes evidence of completion - 5 pts
- Provides 100-150 word narrative - 5 pts
- Compares efficiency to full fine-tuning - 3 pts

3. Deliverable 3: Performance Comparison Analysis

Question Prompt: "Submit a comprehensive analysis comparing your LoRA model against the full fine-tuning baseline. Your analysis must include: (1) quantitative metrics table showing perplexity, ROUGE-L, training time, cost, and memory for both approaches, (2) performance retention percentage calculation, (3) qualitative assessment of at least three generated medical responses evaluating clinical appropriateness and terminology accuracy, and (4) a 250-300 word analysis discussing the trade-offs: where does LoRA match or exceed full fine-tuning, where does it fall short, and what explains the performance gap?"

Response Type: File Upload

File Types: Text (.txt, .md, .csv), Image (.png), Audio (.mp3, .wav), Video (.mp4, .mov - max 15 min)

Grading Criteria:

Includes complete quantitative metrics table - 8 pts

Calculates accurate performance retention - 5 pts

- Provides qualitative assessment of 3+ responses - 8 pts
- Includes 250-300 word trade-off analysis - 6 pts
- Explains performance gap technically - 3 pts

3. Deliverable 3: Production Deployment Recommendation

Question Prompt: "Based on your experimental results, write a data-driven production deployment recommendation (300-400 words or 5-7 minute audio/video) that addresses: (1) your deployment decision (LoRA, full fine-tuning, or hybrid) with specific justification citing your metrics, (2) identification of 2-3 key risks and corresponding mitigation strategies, (3) a phased deployment plan with timeline, (4) specific conditions that would trigger reassessment or switching approaches, and (5) consideration of the startup's constraints ($2K budget, 2-week deadline, iteration needs). Your recommendation should demonstrate executive-level thinking, balancing technical performance with business realities."

Response Type: File Upload

File Types: Text (.txt, .md), Audio (.mp3, .wav, .aac, .flac), Video (.mp4, .mov, .webm - max 15 min)

Grading Criteria:

States a clear decision with metric-based justification - 7 pts

Identifies 2-3 risks with mitigation strategies - 6 pts

- Includes phased deployment plan - 4 pts
- Defines reassessment triggers - 4 pts
- Meets word/time requirements - 4 pts

Additional Information
Best Practices for
AI-Graded Submission

Creating Effective Question Prompts

1. Be Specific: Include specific numbers or criteria in your prompts to help AI generate clear assessment criteria.
2. Include Context: Reference specific elements from the HOL activity.
3. Encourage Depth: Ask for examples, justifications, or connections.
4. Align with Objectives: Ensure prompts assess stated learning objectives.

Submission Requirement Guidelines

1. Clarity: Each requirement should have one clear focus.
2. Balance: Mix different types of submissions (creative, analytical, reflective).
3. Inclusivity: Always provide alternative formats.
4. Progression: Order requirements from concrete to abstract.

File Type Selection Guide

Always offer multiple formats for inclusivity:

Submission Type	Recommended File Options
Written Work	Text, Audio (for verbal responses), Video (for sign language)
Visual Creation	Image, Video, Coding (for programmatic art)
Technical Solution	Coding, Text, Video (for demonstration)
Presentation	Video, Audio, Image (slides), Text (script)
Reflection	Text, Audio, Video

Standard Options

- **Text:** .txt, .text, .csv, .md, .json, .log
- **Image:** .png, .jpg, .jpeg, .jfif, .webp, .bmp, .tiff, .tif
- **Video:** .mp4, .mov, .webm (15 min max)
- **Audio:** .mp3, .wav, .aac, .flac
- **Coding:** .bash, .bat, .c, .cpp, .css, .go, .htm, .html, .java, .json, .jsp, .jsx, .md, .php, .py, .r, .scss, .sh, .sml, .sql, .swift, .ts, .tsx, .xml, .yml, .js

Size Limits:

- Images: 5 MB maximum
- Files: 2 GB maximum
- Videos: 15 minutes maximum

Accessibility Considerations:

- Always allow multiple file types for each requirement.
- Include both video AND audio options for verbal responses.
- Provide text alternatives for visual submissions.

Quality Assurance Checklist

Before finalizing your HOL:

- ✓ All prompts are clear and specific
- ✓ Multiple file types offered for each upload requirement
- ✓ Submission types match the nature of the task
- ✓ Requirements align with learning objectives
- ✓ Instructions are step-by-step and actionable

- ✓ Accessibility options are available throughout
- ✓ Time estimates are realistic and add up to 90 minutes
- ✓ Expected outcomes are provided for each major step
- ✓ Tips and warnings are included where necessary

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.